基于 RISC-V 与 FPGA 异构架构的

2D 图形渲染加速引擎

演示视频讲稿

2D 图形渲染加速引擎 · 边演示边讲解

FPGA 赛道 · 易灵思 Ti60F225 平台

演示视频讲稿（边演示边讲解 · 约 3 分钟）

0:00 – 0:20　开场

【画面：开发板 + 显示器同框，屏幕上是弹幕场景】

「这是我们做的 2D 图形渲染加速引擎，跑在易灵思 Ti60F225 这款 FPGA 上。

左边是 RISC-V 处理器，右边是我们自己写的图形渲染硬件，两边通过 AXI4 总线连在一起。」

「现在屏幕上是一千五百个八乘八像素的精灵，全部由 FPGA 硬件渲染。」

【手指屏幕左上角 HUD】

「左边这行字是实时状态：ELEM 1500 表示同屏精灵数，HW 59.8 是当前帧率——基本就是满帧。

SW 0.0 说明这一帧 CPU 一个像素都没画。」

0:20 – 0:55　创新点一：精灵调度硬件化

「这里有个容易被忽略的问题：光有渲染硬件还不够。」

「如果让 CPU 一个精灵一个精灵地发命令，一千五百个精灵就要发一千五百次，CPU 直接卡死。」

「所以我们把精灵的调度也放进了硬件——精灵表遍历器。

CPU 只在游戏开始时写一次精灵表，之后每帧只发一个启动信号，剩下的遍历、位置更新、擦除、绘制全是硬件自己完成。」

「代价就是刚才那个 SW 0.0——CPU 每帧对精灵的操作次数是零。」

0:55 – 1:25　CPU 渲染对比

【按下 A 键，画面变卡】

「现在我把渲染切到 CPU 纯软件——完全相同的画面，同样的 1500 个精灵，全部用 RISC-V 一个一个像素写。」

【手指 HUD】

「SW 掉到 20 帧左右，肉眼能明显看到卡顿。」

【按下 D 键切回】

「切回硬件渲染，立刻回到 59.8 帧。」

「同样的画面，硬件比软件快三倍左右。」

1:25 – 1:55　Alpha 混合效果

「再仔细看这些精灵——大约一半是半透明的，能透出背后的背景。」

【手指画面上半透明精灵】

「这是 Alpha 混合，也是硬件做的。半透明的精灵在引擎里走的是『读背景、混合、再写回』的通路，

和不透明的精灵是完全不同的硬件路径。」

「引擎还支持颜色键控去背、硬件裁剪这些常规 2D 功能。」

1:55 – 2:40　游戏演示

【按下 G，进入游戏】

「光看压测场景比较枯燥，我们还用这个引擎做了一个弹幕射击游戏。」

「玩家自动开火，我只需要移动。注意左上角的帧率——正常游玩是接近 60 帧。」

【按下 B，一键满级】

「现在按 B 键开极限压力测试：直接升到满级，所有技能拉满，屏幕上瞬间塞满敌人、子弹、宝石、粒子。」

【手指 HUD 数字】

「同屏 360 个对象，帧率 35 帧左右。这是最坏情况下的压力测试数据——

因为每个 16×16 的精灵都要独立读写 DDR，帧存带宽到顶了。」

2:40 – 3:00　收尾

【按 Q 退出游戏，回到弹幕场景】

「总结一下：这套引擎在 1080p 分辨率下做到了同屏 1500 个精灵、59.8 帧，

CPU 每帧对精灵的操作次数是零，同时支持 Alpha 混合、颜色键控和硬件裁剪。」

「我们的核心思路是——不只是把像素搬运交给硬件，把精灵的调度也交给硬件，

让 CPU 真正从图形细节里解放出来。」

「以上，谢谢观看。」

附：讲解时的操作对照表

稿子里的位置	你要说的话	我配合发命令
0:00 开场	介绍平台与 1500 精灵	无（弹幕场景已在跑）
0:55 切 CPU	「切到 CPU 纯软件」	a
1:20 切回	「切回硬件渲染」	d
1:25 Alpha	「一半是半透明的」	无
1:55 进游戏	「做了个弹幕游戏」	g
2:15 满级	「按 B 键极限压力测试」	b
2:40 退出	「总结一下」	q